

Proyecto Completo para Restaurante Pequeño - PostgreSQL 18

Estructura del Proyecto

sql

```
-- =====
-- PROYECTO RESTAURANTE "SABOR CASA"
-- PostgreSQL 18 - Versión Simplificada
-- =====

-- 1. CREACIÓN DE TABLESPACE
-- =====
-- Directorio: /var/lib/postgresql/18/tablespaces/restaurante
CREATE TABLESPACE ts_restaurante
OWNER postgres
LOCATION '/var/lib/postgresql/18/tablespaces/restaurante';

-- 2. CREACIÓN DE ROLES Y USUARIOS
-- =====
-- Rol de administrador
CREATE ROLE admin_restaurante WITH
    LOGIN
    SUPERUSER
    PASSWORD 'Admin123!';

-- Rol para gerente
CREATE ROLE gerente WITH
    LOGIN
    CREATEDB
    PASSWORD 'Gerente123!';

-- Rol para meseros (solo consultas)
CREATE ROLE mesero WITH
    LOGIN
    PASSWORD 'Mesero123!';

-- Rol para cocina (solo insert/update en pedidos)
CREATE ROLE cocina WITH
    LOGIN
    PASSWORD 'Cocina123!';

-- 3. CREACIÓN DE BASE DE DATOS
-- =====
CREATE DATABASE restaurante_db
OWNER admin_restaurante
TABLESPACE ts_restaurante
ENCODING 'UTF8'
LC_COLLATE 'es_ES.UTF-8'
LC_CTYPE 'es_ES.UTF-8'
CONNECTION LIMIT 50;

-- Conectar a la base de datos
\c restaurante_db

-- 4. CREACIÓN DE SCHEMA
```

```

-- =====
CREATE SCHEMA restaurante AUTHORIZATION admin_restaurante;

-- Configurar schema por defecto
SET search_path TO restaurante;

-- 5. CREACIÓN DE TABLAS (Modelo Simplificado)
-- =====

-- Tabla de categorías
CREATE TABLE categorias (
    id_categoria SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion TEXT,
    activo BOOLEAN DEFAULT true
);

-- Tabla de productos/menú
CREATE TABLE productos (
    id_producto SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2) NOT NULL CHECK (precio > 0),
    id_categoria INT REFERENCES categorias(id_categoria),
    stock INT DEFAULT 0,
    activo BOOLEAN DEFAULT true
);

-- Tabla de clientes
CREATE TABLE clientes (
    id_cliente SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    telefono VARCHAR(15),
    email VARCHAR(100),
    fecha_registro DATE DEFAULT CURRENT_DATE
);

-- Tabla de mesas
CREATE TABLE mesas (
    id_mesa SERIAL PRIMARY KEY,
    numero_mesa INT UNIQUE NOT NULL,
    capacidad INT NOT NULL,
    ubicacion VARCHAR(50)
);

-- Tabla de empleados
CREATE TABLE empleados (
    id_empleado SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    rol VARCHAR(50) CHECK (rol IN ('gerente', 'mesero', 'cocina', 'admin')),
    telefono VARCHAR(15),
    fecha_contrato DATE DEFAULT CURRENT_DATE
);

-- Tabla de pedidos
CREATE TABLE pedidos (
    id_pedido SERIAL PRIMARY KEY,
    id_mesa INT REFERENCES mesas(id_mesa),
    id_cliente INT REFERENCES clientes(id_cliente),
    id_empleado INT REFERENCES empleados(id_empleado),
    fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```

        estado VARCHAR(20) DEFAULT 'pendiente' CHECK (estado IN ('pendiente',
'preparando', 'listo', 'entregado', 'cancelado')),
        total DECIMAL(10,2) DEFAULT 0
    );

-- Tabla de detalles del pedido
CREATE TABLE detalles_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT REFERENCES pedidos(id_pedido) ON DELETE CASCADE,
    id_producto INT REFERENCES productos(id_producto),
    cantidad INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario)
STORED
);

-- Tabla de pagos
CREATE TABLE pagos (
    id_pago SERIAL PRIMARY KEY,
    id_pedido INT REFERENCES pedidos(id_pedido),
    monto DECIMAL(10,2) NOT NULL,
    metodo_pago VARCHAR(20) CHECK (metodo_pago IN ('efectivo', 'tarjeta',
'transferencia')),
    fecha_pago TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 6. ÍNDICES PARA RENDIMIENTO
-- =====
CREATE INDEX idx_pedidos_fecha ON pedidos(fecha_pedido);
CREATE INDEX idx_pedidos_estado ON pedidos(estado);
CREATE INDEX idx_detalle_pedido ON detalles_pedido(id_pedido);
CREATE INDEX idx_productos_categoria ON productos(id_categoria);
CREATE INDEX idx_pagos_pedido ON pagos(id_pedido);

-- 7. PERMISOS Y ROLES (Seguridad)
-- =====

-- Permisos para GERENTE (todo excepto eliminar)
GRANT USAGE ON SCHEMA restaurante TO gerente;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA restaurante TO gerente;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA restaurante TO gerente;

-- Permisos para MESERO (solo lectura y crear pedidos)
GRANT USAGE ON SCHEMA restaurante TO mesero;
GRANT SELECT ON mesas, productos, clientes TO mesero;
GRANT SELECT, INSERT ON pedidos, detalles_pedido TO mesero;
GRANT UPDATE ON pedidos TO mesero;
GRANT USAGE ON SEQUENCES pedidos_id_pedido_seq, detalles_pedido_id_detalle_seq
TO mesero;

-- Permisos para COCINA (solo ver y actualizar estado de pedidos)
GRANT USAGE ON SCHEMA restaurante TO cocina;
GRANT SELECT ON pedidos, detalles_pedido, productos TO cocina;
GRANT UPDATE (estado) ON pedidos TO cocina;

-- 8. INSERCIÓN DE DATOS (Información Inicial)
-- =====

-- Insertar categorías
INSERT INTO categorias (nombre, descripcion) VALUES
('Entradas', 'Platos para compartir al inicio'),

```

```

('Platos Principales', 'Nuestros mejores platos'),
('Bebidas', 'Refrescos y jugos naturales'),
('Postres', 'Dulces caseros');

-- Insertar productos
INSERT INTO productos (nombre, precio, id_categoria, stock) VALUES
('Ensalada Mixta', 5.50, 1, 50),
('Sopa de Tomate', 4.00, 1, 40),
('Lomo Saltado', 12.00, 2, 30),
('Pollo a la Plancha', 10.00, 2, 35),
('Coca Cola 500ml', 2.50, 3, 100),
('Jugo Natural', 3.50, 3, 60),
('Torta de Chocolate', 4.50, 4, 25),
('Flan Casero', 3.50, 4, 30);

-- Insertar mesas
INSERT INTO mesas (numero_mesa, capacidad, ubicacion) VALUES
(1, 4, 'Interior'),
(2, 4, 'Interior'),
(3, 6, 'Terraza'),
(4, 2, 'Barra'),
(5, 8, 'Salón Privado');

-- Insertar empleados
INSERT INTO empleados (nombre, rol, telefono) VALUES
('Carlos López', 'gerente', '555-0101'),
('Ana Martínez', 'mesero', '555-0102'),
('Luis Fernández', 'cocina', '555-0103'),
('María García', 'mesero', '555-0104');

-- Insertar clientes
INSERT INTO clientes (nombre, telefono, email) VALUES
('Juan Pérez', '555-1001', 'juan@email.com'),
('Claudia Ríos', '555-1002', 'claudia@email.com'),
('Roberto Díaz', '555-1003', 'roberto@email.com');

-- Insertar pedido de ejemplo
INSERT INTO pedidos (id_mesa, id_cliente, id_empleado, estado)
VALUES (1, 1, 2, 'entregado');

-- Insertar detalles del pedido
INSERT INTO detalles_pedido (id_pedido, id_producto, cantidad, precio_unitario)
VALUES
(1, 3, 2, 12.00), -- 2 Lomo Saltado
(1, 5, 3, 2.50); -- 3 Coca Colas

-- Actualizar total del pedido
UPDATE pedidos SET total = (
    SELECT SUM(subtotal) FROM detalles_pedido WHERE id_pedido = 1
) WHERE id_pedido = 1;

-- Insertar pago
INSERT INTO pagos (id_pedido, monto, metodo_pago)
VALUES (1, 31.50, 'efectivo');

-- 9. FUNCIONES Y TRIGGERS (Automatización)
-- =====

-- Función para actualizar stock automáticamente
CREATE OR REPLACE FUNCTION actualizar_stock()
RETURNS TRIGGER AS $$

```

```

BEGIN
    UPDATE productos
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger que actualiza stock al crear detalle de pedido
CREATE TRIGGER trg_actualizar_stock
AFTER INSERT ON detalles_pedido
FOR EACH ROW
EXECUTE FUNCTION actualizar_stock();

-- Función para calcular total del pedido
CREATE OR REPLACE FUNCTION calcular_total_pedido()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE pedidos
    SET total = (
        SELECT SUM(subtotal)
        FROM detalles_pedido
        WHERE id_pedido = NEW.id_pedido
    )
    WHERE id_pedido = NEW.id_pedido;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para recalcular total
CREATE TRIGGER trg_calcular_total
AFTER INSERT OR UPDATE ON detalles_pedido
FOR EACH ROW
EXECUTE FUNCTION calcular_total_pedido();

-- 10. VISTAS PARA ADMINISTRACIÓN
-- =====

-- Vista de ventas del día
CREATE VIEW ventas_hoy AS
SELECT
    p.id_pedido,
    c.nombre AS cliente,
    m.numero_mesa,
    p.total,
    p.fecha_pedido
FROM pedidos p
LEFT JOIN clientes c ON p.id_cliente = c.id_cliente
JOIN mesas m ON p.id_mesa = m.id_mesa
WHERE DATE(p.fecha_pedido) = CURRENT_DATE;

-- Vista de productos más vendidos
CREATE VIEW productos_top AS
SELECT
    pr.nombre,
    SUM(dp.cantidad) AS total_vendido,
    COUNT(DISTINCT dp.id_pedido) AS num_pedidos
FROM detalles_pedido dp
JOIN productos pr ON dp.id_producto = pr.id_producto
GROUP BY pr.nombre
ORDER BY total_vendido DESC

```

```

LIMIT 10;

-- 11. MONITOREO Y MANTENIMIENTO
-- =====

-- Ver tamaño de la base de datos
SELECT
    pg_database_size('restaurante_db')/1024/1024 AS size_mb,
    pg_database_size('restaurante_db')/1024/1024/1024 AS size_gb;

-- Ver tablas más grandes
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size
FROM pg_tables
WHERE schemaname = 'restaurante'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- Índices no utilizados (para optimizar)
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan AS number_of_scans
FROM pg_stat_user_indexes
WHERE idx_scan = 0
AND schemaname = 'restaurante';

-- Ver queries lentas (requiere habilitar logging)
-- Configurar en postgresql.conf:
-- log_min_duration_statement = 10000 (ms)

-- 12. FUNCIONES DE ADMINISTRACIÓN
-- =====

-- Backup automático (comando desde terminal)
-- pg_dump -U admin_restaurante -h localhost -F c -b -v -f "backup_restaurante_$
(date +%Y%m%d).dump" restaurante_db

-- Vacuum y Analyze (mantenimiento)
VACUUM ANALYZE restaurante.pedidos;
VACUUM ANALYZE restaurante.detalles_pedido;

-- Reindexar tablas problemáticas
REINDEX TABLE restaurante.pedidos;
REINDEX TABLE restaurante.detalles_pedido;

-- 13. REPORTE DE EJEMPLO
-- =====

-- Reporte de ventas por mes
CREATE OR REPLACE FUNCTION reporte_ventas_mensual(anio INT, mes INT)
RETURNS TABLE(
    dia DATE,
    total_ventas DECIMAL,
    num_pedidos BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT

```

```

        DATE(p.fecha_pedido) AS dia,
        SUM(p.total) AS total_ventas,
        COUNT(*) AS num_pedidos
FROM pedidos p
WHERE EXTRACT(YEAR FROM p.fecha_pedido) = anio
      AND EXTRACT(MONTH FROM p.fecha_pedido) = mes
      AND p.estado = 'entregado'
GROUP BY DATE(p.fecha_pedido)
ORDER BY dia;
END;
$$ LANGUAGE plpgsql;

-- Uso: SELECT * FROM reporte_ventas_mensual(2026, 5);

-- 14. CONSULTAS ÚTILES PARA EL DÍA A DÍA
-- =====

-- Pedidos activos (para cocina)
SELECT
    p.id_pedido,
    m.numero_mesa,
    pr.nombre AS producto,
    dp.cantidad,
    p.estado
FROM pedidos p
JOIN mesas m ON p.id_mesa = m.id_mesa
JOIN detalles_pedido dp ON p.id_pedido = dp.id_pedido
JOIN productos pr ON dp.id_producto = pr.id_producto
WHERE p.estado IN ('pendiente', 'preparando')
ORDER BY p.fecha_pedido;

-- Facturación diaria
SELECT
    DATE(fecha_pago) AS dia,
    COUNT(*) AS num_pagos,
    SUM(monto) AS total,
    AVG(monto) AS promedio
FROM pagos
WHERE DATE(fecha_pago) = CURRENT_DATE
GROUP BY DATE(fecha_pago);

-- Control de inventario (productos con bajo stock)
SELECT
    nombre,
    stock,
    CASE
        WHEN stock < 5 THEN 'CRÍTICO - Comprar urgente'
        WHEN stock < 10 THEN 'BAJO - Revisar inventario'
        ELSE 'Normal'
    END AS estado_stock
FROM productos
WHERE stock < 10
ORDER BY stock;

-- =====
-- FIN DEL PROYECTO
-- =====

-- Para eliminar todo (NUNCA EN PRODUCCIÓN)
-- DROP DATABASE restaurante_db;
-- DROP TABLESPACE ts_restaurante;

```

```
-- DROP OWNED BY admin_restaurante CASCADE;  
-- DROP ROLE admin_restaurante, gerente, mesero, cocina;
```

Pasos para Implementar

1. Requisitos previos:

```
bash  
  
# Instalar PostgreSQL 18  
sudo apt install postgresql-18  
  
# Crear directorio para tablespace  
sudo mkdir -p /var/lib/postgresql/18/tablespaces/restaurante  
sudo chown postgres:postgres /var/lib/postgresql/18/tablespaces/restaurante
```

2. Ejecutar el script:

```
bash  
  
# Conectarse como superusuario  
sudo -u postgres psql  
  
# Copiar todo el script y pegarlo en psql  
\i 'ruta/al/archivo/restaurante.sql'
```

3. Pruebas rápidas:

```
sql  
  
-- Conectar como mesero  
\c restaurante_db mesero  
  
-- Ver menú  
SELECT * FROM productos WHERE activo = true;  
  
-- Crear nuevo pedido  
INSERT INTO pedidos (id_mesa, id_empleado) VALUES (3, 2);  
INSERT INTO detalles_pedido (id_pedido, id_producto, cantidad, precio_unitario)  
VALUES (currval('pedidos_id_pedido_seq'), 4, 2, 10.00);
```

Monitoreo Diario

```
sql  
  
-- Ver actividad actual  
SELECT  
    pid,  
    username,  
    application_name,  
    state,  
    query  
FROM pg_stat_activity  
WHERE datname = 'restaurante_db';
```

```
-- Ver conexiones activas  
SELECT COUNT(*) FROM pg_stat_activity WHERE datname = 'restaurante_db';
```

Este proyecto incluye todo lo necesario para un restaurante pequeño: base de datos, gestión de roles, automatización con triggers, vistas de negocio, y scripts de mantenimiento. ¡Todo funcionando desde el primer momento!

Esta respuesta es generada por AI, solo como referencia.

explicame esta línea :REINDEX TABLE restaurante.pedidos;

Explicación detallada de REINDEX TABLE restaurante.pedidos;

¿Qué hace esta línea?

Reconstruye todos los índices de la tabla pedidos en el esquema restaurante, eliminando los índices antiguos y creando versiones nuevas y optimizadas.

Conceptos Fundamentales

¿Qué son los índices?

Los índices en PostgreSQL son como el **índice de un libro**:

- Te permiten encontrar información rápidamente sin leer todo el libro
- En la tabla pedidos, tenemos índices en fecha_pedido y estado

¿Por qué se degradan los índices con el tiempo?

sql

```
-- Ejemplo práctico de lo que pasa
-- Situación inicial: Índice eficiente
Buscar pedido #1000 → Usa índice → Rápido (0.1 seg)

-- Después de muchas operaciones (inserts, updates, deletes)
-- El índice se vuelve "fragmentado" como un disco duro desorganizado
Buscar pedido #1000 → Usa índice fragmentado → Lento (1.5 seg)

-- Después de REINDEX
Buscar pedido #1000 → Usa índice nuevo → Rápido (0.1 seg)
```

Anatomía de la línea

sql

```
REINDEX TABLE restaurante.pedidos;
--      |           |           |
--      |           |           | Nombre de la tabla
--      |           |           | Esquema donde está la tabla
--      |           |           | Comando para reconstruir índices
```

¿Qué índices se reconstruyen exactamente?

sql

```
-- Los índices que se crearon originalmente en la tabla pedidos:
CREATE INDEX idx_pedidos_fecha ON pedidos(fecha_pedido);
CREATE INDEX idx_pedidos_estado ON pedidos(estado);
```

```
-- REINDEX reconstruye AMBOS índices automáticamente
-- También reconstruye índices internos como la clave primaria (id_pedido)
```

Ejemplo visual del problema

Antes de REINDEX (fragmentado):

text

```
Índice de fecha_pedido:
[2024-01][hueco][2024-02][hueco][hueco][2024-01][2024-03][hueco]
                                     ↑
                               Datos desordenados
```

Después de REINDEX (optimizado):

text

```
Índice de fecha_pedido:
[2024-01][2024-01][2024-02][2024-02][2024-03][2024-03]
                ↑
            Todo ordenado y compacto
```

Cuándo usar REINDEX

DEBES usarlo cuando:

sql

```
-- 1. Los índices están corruptos (raro pero pasa)
-- 2. Notas que las consultas se vuelven lentas gradualmente
-- 3. Después de eliminar MUCHOS registros (ej: limpieza masiva)
-- 4. Como mantenimiento mensual/trimestral

-- Señales de alerta:
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,      -- Número de veces usado
    idx_tup_read,  -- Tuplas leídas
    idx_tup_fetch  -- Tuplas recuperadas
FROM pg_stat_user_indexes
WHERE tablename = 'pedidos';
-- Si idx_scan es cero, el índice no se usa (quizás está corrupto o inútil)
```

NO debes usarlo cuando:

sql

```
-- 1. Hay muchos usuarios conectados (puede bloquear la tabla)
-- 2. En horario pico (11:00-15:00, 19:00-23:00 para un restaurante)
-- 3. Demasiado frecuente (diario es excesivo para un restaurante pequeño)
```

Bloqueo y rendimiento

Niveles de bloqueo:

sql

```
-- REINDEX normal (BLOQUEA LA TABLA POR COMPLETO)
REINDEX TABLE restaurante.pedidos;
-- Durante la operación: NADIE puede leer ni escribir en pedidos
-- Duración típica: 1-5 segundos para tablas pequeñas

-- Alternativa CONCURRENT (PostgreSQL 12+)
REINDEX TABLE CONCURRENTLY restaurante.pedidos;
-- Permite lectura/escritura durante la operación
-- PERO: Toma más tiempo (5-10 segundos)
```

Ejemplo práctico en un restaurante

sql

```
-- Escenario: 3 meses de operación, 5000 pedidos
-- Los índices se han fragmentado por actualizaciones frecuentes de estado

-- ANTES del REINDEX:
EXPLAIN ANALYZE SELECT * FROM pedidos
WHERE fecha_pedido > '2026-05-01' AND estado = 'entregado';
-- Tiempo: 850 ms

-- Ejecutar mantenimiento (ideal a las 3 AM)
REINDEX TABLE restaurante.pedidos;

-- DESPUÉS del REINDEX:
-- Tiempo: 120 ms      (7 veces más rápido!)
```

Formas alternativas de REINDEX

sql

```
-- 1. Reindexar un índice específico
REINDEX INDEX idx_pedidos_fecha;

-- 2. Reindexar TODO el esquema
REINDEX SCHEMA restaurante;

-- 3. Reindexar TODO el sistema (raro en producción)
REINDEX DATABASE restaurante_db;

-- 4. Verificar si necesita REINDEX
SELECT
    relname,
    pg_size_pretty(pg_relation_size(oid)) as index_size,
    pg_stat_get_blocks_fetched(oid) as blocks_fetched,
    pg_stat_get_blocks_hit(oid) as blocks_hit
FROM pg_class
WHERE relname LIKE 'idx_pedidos%';
-- Si blocks_hit es bajo (< 90%), necesita REINDEX
```

Mejores prácticas para tu restaurante

sql

```
-- Crear script de mantenimiento semanal
CREATE OR REPLACE FUNCTION mantenimiento_semanal()
RETURNS TEXT AS $$
DECLARE
    inicio TIMESTAMP;
```

```

BEGIN
    inicio := clock_timestamp();

    -- Para tablas pequeñas (< 10,000 registros)
    REINDEX TABLE restaurante.pedidos;
    REINDEX TABLE restaurante.detalles_pedido;

    -- Vacuum para limpiar espacio
    VACUUM ANALYZE restaurante.pedidos;

    RETURN format('Mantenimiento completado en %s',
                  clock_timestamp() - inicio);
END;
$$ LANGUAGE plpgsql;

-- Programar con pg_cron (extensión)
-- Ejecutar cada domingo a las 3:00 AM
SELECT cron.schedule('maintenance', '0 3 * * 0',
                     'SELECT mantenimiento_semanal();');

```

Resumen

REINDEX TABLE restaurante.pedidos; =

- **Herramienta de mantenimiento** que optimiza índices
- **Mejora rendimiento** de consultas hasta 7x
- **Hacerlo** semanal/mensualmente, en horas de poco uso
- **Cuidado** con el bloqueo en horas pico
- **Esencial** para mantener la base de datos "como nueva"

Para un restaurante pequeño, con ejecutarlo **una vez al mes o después de limpiar pedidos antiguos** es más que suficiente.

porque no sacas en detalles_pedido el precio del producto. Tenemos que memorizar el precio del producto?

¡Excelente observación!

El problema que señalas es REAL y muy importante

En mi diseño original, detalles_pedido tiene precio_unitario como un campo que se debe llenar manualmente:

sql

```

-- Mi diseño original (CON problema)
CREATE TABLE detalles_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT REFERENCES pedidos(id_pedido),
    id_producto INT REFERENCES productos(id_producto),
    cantidad INT NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL, -- Hay que escribirlo manualmente

```

```

        subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario)
STORED
);

```

¿Por qué es un problema?

sql

```

-- Escenario del mundo real:
-- 1. El precio del Lomo Saltado sube de $12 a $14
UPDATE productos SET precio = 14 WHERE nombre = 'Lomo Saltado';

-- 2. Problema: Los pedidos ANTIGUOS ahora muestran precio incorrecto
SELECT * FROM detalles_pedido WHERE id_producto = 3;
-- El precio_unitario sigue siendo $12 (el precio al momento del pedido)
-- PERO si consultamos el precio actual del producto es $14

-- 3. ¡Registro histórico incorrecto!
-- No sabemos si ese pedido se pagó a $12 o a $14

```

La solución correcta (2 enfoques)

Enfoque 1: Guardar precio histórico (RECOMENDADO)

sql

```

-- Guardar el precio del momento del pedido usando TRIGGER
CREATE TABLE detalles_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT REFERENCES pedidos(id_pedido),
    id_producto INT REFERENCES productos(id_producto),
    cantidad INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10,2), -- Puede ser NULL si no se especifica
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario)
STORED
);

-- TRIGGER que captura el precio actual automáticamente
CREATE OR REPLACE FUNCTION obtenerPrecioProducto()
RETURNS TRIGGER AS $$
BEGIN
    -- Si no se especificó precio_unitario, usar el precio actual
    IF NEW.precio_unitario IS NULL THEN
        SELECT precio INTO NEW.precio_unitario
        FROM productos
        WHERE id_producto = NEW.id_producto;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_precio_producto
BEFORE INSERT ON detalles_pedido
FOR EACH ROW
EXECUTE FUNCTION obtenerPrecioProducto();

-- Ahora puedes insertar sin especificar precio
INSERT INTO detalles_pedido (id_pedido, id_producto, cantidad)
VALUES (1, 3, 2); -- Automáticamente captura el precio actual ($12)

```

Enfoque 2: No guardar precio (SIMPLE pero limitado)

sql

```
-- Sin campo precio_unitario, calcular todo en tiempo real
CREATE TABLE detalles_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT REFERENCES pedidos(id_pedido),
    id_producto INT REFERENCES productos(id_producto),
    cantidad INT NOT NULL,
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * (SELECT precio FROM
productos WHERE id_producto = id_producto)) STORED -- NO FUNCIONA así
);

-- En su lugar, usar VISTA
CREATE VIEW v_detalle_pedido AS
SELECT
    dp.id_detalle,
    dp.id_pedido,
    dp.id_producto,
    dp.cantidad,
    p.precio AS precio_actual,
    dp.cantidad * p.precio AS subtotal
FROM detalles_pedido dp
JOIN productos p ON dp.id_producto = p.id_producto;

-- PROBLEMA: Si el precio cambia, los pedidos antiguos se recalcularían mal
```

La solución profesional (usada en sistemas reales)

sql

```
-- Tabla de precios históricos (lo más profesional)
CREATE TABLE precios_historicos (
    id_precio SERIAL PRIMARY KEY,
    id_producto INT REFERENCES productos(id_producto),
    precio DECIMAL(10,2) NOT NULL,
    fecha_inicio DATE NOT NULL DEFAULT CURRENT_DATE,
    fecha_fin DATE DEFAULT '9999-12-31'
);

-- Tabla de detalles de pedido CORRECTA
CREATE TABLE detalles_pedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT REFERENCES pedidos(id_pedido),
    id_producto INT REFERENCES productos(id_producto),
    id_precio INT REFERENCES precios_historicos(id_precio), -- ← Clave histórica
    cantidad INT NOT NULL,
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * (SELECT precio FROM
precios_historicos WHERE id_precio = id_precio)) STORED
);

-- Trigger para asignar automáticamente el precio histórico correcto
CREATE OR REPLACE FUNCTION asignar_precio_historico()
RETURNS TRIGGER AS $$
DECLARE
    v_precio_id INT;
BEGIN
    -- Buscar el precio vigente en la fecha del pedido
    SELECT ph.id_precio INTO v_precio_id
    FROM precios_historicos ph
```

```

JOIN pedidos p ON p.id_pedido = NEW.id_pedido
WHERE ph.id_producto = NEW.id_producto
      AND p.fecha_pedido BETWEEN ph.fecha_inicio AND COALESCE(ph.fecha_fin,
'9999-12-31')
LIMIT 1;

IF v_precio_id IS NULL THEN
  -- Si no hay histórico, usar el precio actual
  INSERT INTO precios_historicos (id_producto, precio, fecha_inicio)
  VALUES (NEW.id_producto, (SELECT precio FROM productos WHERE id_producto
= NEW.id_producto), CURRENT_DATE)
  RETURNING id_pedido INTO v_precio_id;
END IF;

NEW.id_precio := v_precio_id;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_asignar_precio_historico
BEFORE INSERT ON detalles_pedido
FOR EACH ROW
EXECUTE FUNCTION asignar_precio_historico();

-- Manejo de cambios de precio (ejemplo)
-- Cuando sube el precio del Lomo Saltado:
UPDATE precios_historicos
SET fecha_fin = '2026-05-07'
WHERE id_producto = 3 AND fecha_fin = '9999-12-31';

INSERT INTO precios_historicos (id_producto, precio, fecha_inicio)
VALUES (3, 14.00, '2026-05-08');

```

Comparativa de soluciones

Enfoque	Ventajas	Desventajas	Uso recomendado
Campo manual (mi original)	Simple, flexible	Error humano, inconsistencia	No recomendar
Trigger automático	Sencillo, confiable	No guarda historial de precios	Restaurante pequeño
Tabla histórica	Perfecto, auditoría completa	Más complejo	Restaurante grande

Solución SIMPLIFICADA para tu restaurante pequeño

sql

```

-- La mejor relación calidad-simplicidad
CREATE TABLE detalles_pedido (
  id_detalle SERIAL PRIMARY KEY,
  id_pedido INT REFERENCES pedidos(id_pedido),
  id_producto INT REFERENCES productos(id_producto),
  cantidad INT NOT NULL,
  precio_unitario DECIMAL(10,2) NOT NULL, -- Se llena automáticamente
  subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario)
STORED
);

```

```

-- Trigger simple y efectivo
CREATE OR REPLACE FUNCTION auto_precio()
RETURNS TRIGGER AS $$
BEGIN
    SELECT precio INTO NEW.precio_unitario
    FROM productos
    WHERE id_producto = NEW.id_producto;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER auto_precio_trigger
BEFORE INSERT ON detalles_pedido
FOR EACH ROW
EXECUTE FUNCTION auto_precio();

-- Ahora puedes insertar así de simple:
INSERT INTO detalles_pedido (id_pedido, id_producto, cantidad)
VALUES (1, 3, 2); -- El precio se asigna automáticamente

-- Y consultar así:
SELECT
    dp.id_detalle,
    p.nombre AS producto,
    dp.cantidad,
    dp.precio_unitario, -- Precio al momento del pedido
    dp.subtotal
FROM detalles_pedido dp
JOIN productos p ON dp.id_producto = p.id_producto;

```

Lección aprendida

NO debes obligar al usuario a recordar/memorizar el precio. El sistema debe:

1. **Capturar automáticamente** el precio actual
2. **Congelar el precio** en el momento del pedido
3. **Permitir cambios** de precio sin afectar pedidos históricos

¡Gracias por la observación! Esto mejora MUCHO el diseño original.